



EDUCAÇÃO A DISTÂNCIA

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Unidade 3 | Semana 5

Sistema de memória e dispositivos de entrada e saída



INSTITUTO FEDERAL
Sudeste de Minas Gerais

Educação a Distância
Pró-reitoria de Ensino

Curso Técnico em Desenvolvimento de Sistemas

Professor: Jeronimo Costa Penha

Apostila V1 | 2026



Apresentação

Na Semana 5, o foco passa do processamento das instruções para os locais onde dados e programas permanecem e para os caminhos usados na comunicação com o mundo externo. Nenhum computador depende de uma única memória: registradores, cache, memória principal e armazenamento secundário formam uma hierarquia que equilibra velocidade, capacidade, custo e permanência.

Também estudaremos como teclado, tela, unidades de armazenamento, rede e outros periféricos trocam dados com o processador. Essa comunicação é organizada por controladores e pode usar consulta periódica, interrupções ou acesso direto à memória.

Ideia central: o desempenho percebido por um programa depende tanto do processador quanto da movimentação de dados entre memória, armazenamento e dispositivos de entrada e saída.

Objetivos de aprendizagem

- Explicar por que os computadores utilizam uma hierarquia de memória.
- Comparar registradores, cache, RAM e armazenamento secundário.
- Distinguir memória volátil de memória não volátil e reconhecer DRAM, SRAM e ROM.
- Relacionar endereço, capacidade, leitura e escrita na memória principal.
- Identificar dispositivos de entrada, saída e entrada e saída.
- Explicar o papel de controladores, interfaces, interrupções e DMA.
- Relacionar esses conceitos ao desempenho e ao comportamento de sistemas de software.

Roteiro de estudos

1. Leia as seções 1 a 8 e construa um quadro comparando os níveis da hierarquia.
2. Assista ao primeiro vídeo e revise os exemplos de memória e armazenamento.
3. Leia as seções 9 a 15 e acompanhe os fluxos de entrada e saída.
4. Assista ao segundo vídeo e resolva os exercícios de consolidação.
5. Realize o questionário formativo e revise os itens comentados.

Atividade	Tempo
Leitura da apostila	1h30
Dois vídeos	25 min
Exemplos e exercícios	50 min
Questionário formativo	35 min

1. Por que existe uma hierarquia de memória

O processador executa operações em intervalos muito curtos, mas nem toda tecnologia de armazenamento consegue fornecer dados na mesma velocidade. Memórias extremamente rápidas são caras e costumam oferecer pouca capacidade; tecnologias de grande capacidade são mais baratas por bit, porém apresentam maior latência.

A arquitetura resolve esse conflito combinando níveis. Os dados mais usados permanecem próximos do processador; conjuntos maiores ficam em níveis mais lentos e mais baratos. O sistema tenta fazer com que o nível superior encontre o que precisa sem consultar constantemente o nível inferior.

Quatro critérios para comparar memórias

Critério	Pergunta prática	Tendência na hierarquia
Velocidade	Quanto tempo é necessário para obter ou gravar um dado?	Maior perto da CPU
Capacidade	Quantos bytes podem ser mantidos?	Maior nos níveis inferiores
Custo por bit	Quanto custa armazenar cada bit?	Menor nos níveis inferiores
Volatilidade	O conteúdo permanece sem energia?	Armazenamento tende a ser não volátil

Latência e taxa de transferência

Latência é o intervalo entre solicitar um dado e poder utilizá-lo. Taxa de transferência, ou largura de banda, indica quanto pode ser transferido por unidade de tempo. Um dispositivo pode ter boa taxa para blocos grandes e ainda apresentar latência perceptível em pequenos acessos aleatórios.

Exemplo: abrir milhares de arquivos pequenos pode ser mais lento do que ler um único arquivo grande com o mesmo volume total. No primeiro caso, várias solicitações pagam repetidamente o custo de localizar e iniciar cada acesso.

Princípio de localidade

Programas costumam reutilizar dados e instruções acessados recentemente e também acessar posições próximas. Essas tendências são chamadas de localidade temporal e localidade espacial. Elas explicam por que níveis pequenos e rápidos podem atender uma parte elevada dos acessos.

Nesta semana, a cache aparece como um nível da hierarquia. Os mecanismos de acerto, falha, mapeamento e substituição serão aprofundados na Semana 6.

2. Níveis da hierarquia de memória



Figura 1 — Da menor latência e menor capacidade para a maior capacidade e maior latência.

Nível	Função	Características usuais	Exemplo
Registradores	Manter operandos, endereços e estado imediato da execução	Muito rápidos, pouquíssima capacidade, voláteis	PC, IR e registradores gerais
Cache	Guardar cópias de blocos usados com frequência	Rápida, pequena, volátil	L1, L2 e L3
Memória principal	Manter programas e dados ativos	Capacidade intermediária, volátil	RAM do computador
Armazenamento secundário	Conservar arquivos e programas	Grande capacidade, não volátil, maior latência	SSD, HDD e cartão de memória

Movimentação em blocos

Os níveis não trocam necessariamente um único byte por vez. Cache e memória principal, por exemplo, movimentam blocos. Armazenamento secundário também trabalha com unidades maiores, como setores e páginas. Isso aproveita a localidade espacial e reduz o custo de iniciar cada transferência.

Quem controla cada movimento

- **Registradores:** as instruções indicam explicitamente quais registradores serão lidos ou escritos.
- **Cache:** o hardware decide automaticamente se o dado solicitado está disponível nesse nível.
- **Memória principal:** processador e controladores emitem operações de leitura e escrita por endereços.
- **Armazenamento:** o sistema operacional e o controlador do dispositivo organizam operações em blocos.

A hierarquia não transforma armazenamento em memória principal. Um arquivo executável precisa ter suas instruções e seus dados trazidos para a RAM antes que a CPU possa executá-los diretamente.

3. Volatilidade e tipos de memória

Uma memória é volátil quando perde o conteúdo ao deixar de receber energia. Registradores, cache e RAM são voláteis. SSDs, HDDs, memória flash e ROM são não voláteis: preservam dados mesmo com o equipamento desligado.

RAM e ROM não são nomes equivalentes

RAM designa uma memória de acesso aleatório usada principalmente para leitura e escrita durante a execução. ROM designa uma família de memórias não voláteis destinadas a guardar conteúdo que muda pouco, como firmware. Em sistemas atuais, firmware costuma ser mantido em tecnologias regraváveis, embora a sigla ROM continue sendo usada no contexto arquitetural.

Tecnologia	Como armazena	Uso típico	Observação
DRAM	Carga elétrica que precisa ser renovada	Memória principal	Alta densidade e custo menor por bit
SRAM	Circuitos biestáveis sem renovação periódica	Cache	Mais rápida e mais cara por bit
ROM	Conteúdo não volátil	Firmware e inicialização	Leitura frequente e alteração rara
Flash	Células não voláteis regraváveis	SSD, pendrive, cartão e firmware	Escrita ocorre em unidades e tem desgaste limitado

DRAM e atualização do conteúdo

Na DRAM, cada bit depende de uma carga que se dissipa. Por isso, o circuito precisa atualizar periodicamente as células, operação conhecida como refresh. O termo dinâmico vem dessa necessidade. A atualização é administrada pelo subsistema de memória e não pelo programa da aplicação.

SRAM e proximidade do processador

A SRAM não precisa do mesmo refresh, responde mais rapidamente, mas utiliza mais componentes por bit. Ela é adequada a estruturas pequenas e rápidas, como cache. Isso ilustra novamente o compromisso entre velocidade, capacidade e custo.

Cuidado conceitual: memória principal não é sinônimo de armazenamento. A RAM mantém o estado ativo do programa; SSD ou HDD conserva arquivos de forma persistente.

4. Endereços, capacidade e operações

A memória principal pode ser vista como um conjunto de posições identificadas por endereços. Quando a arquitetura é endereçável por byte, cada endereço seleciona um byte. Instruções podem ler ou escrever grupos maiores, como palavras de 16, 32 ou 64 bits, utilizando posições consecutivas.

Relação entre bits de endereço e posições

Com **n** bits de endereço, podem ser representadas **2ⁿ** combinações. Se cada endereço identifica um byte, o espaço endereçável contém **2ⁿ bytes**. O cálculo descreve o alcance dos endereços; a quantidade física instalada pode ser menor.

Bits de endereço	Posições possíveis	Capacidade se cada posição tiver 1 byte
8	$2^8 = 256$	256 bytes
16	$2^{16} = 65\,536$	64 KiB
32	2^{32}	4 GiB
64	2^{64}	16 EiB de espaço teórico

Exemplo resolvido

Considere uma memória endereçável por byte com 12 bits de endereço. Ela possui $2^{12} = 4\,096$ posições, portanto pode endereçar 4 096 bytes, equivalentes a 4 KiB. Os endereços variam de 0 a 4 095.

Leitura e escrita

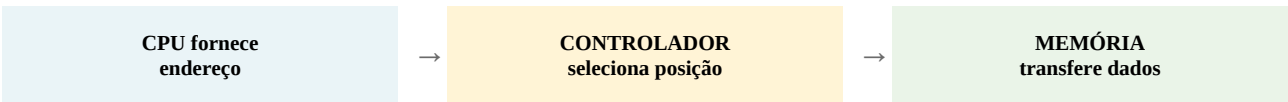


Figura 2 — Visão simplificada de uma operação de leitura.

Na leitura, a memória devolve o conteúdo da posição solicitada. Na escrita, a CPU ou outro controlador fornece o endereço e o valor a ser armazenado. Sinais de controle distinguem as duas operações.

Unidades de capacidade

Em contexto binário, 1 KiB = 1 024 bytes, 1 MiB = 1 024 KiB e 1 GiB = 1 024 MiB. Fabricantes de armazenamento também usam prefixos decimais: 1 GB = 1 000 000 000 bytes. A diferença entre GB e GiB ajuda a explicar por que a capacidade exibida pelo sistema pode parecer menor que o valor comercial.

5. Armazenamento secundário

Armazenamento secundário conserva programas, documentos, bancos de dados e arquivos do sistema. Sua capacidade é muito maior que a da RAM, mas o acesso apresenta maior latência. Dados precisam ser transferidos para a memória principal antes de serem processados pela CPU.

Tecnologia	Princípio	Pontos fortes	Limitações
HDD	Discos magnéticos e cabeças móveis	Grande capacidade e baixo custo por gigabyte	Latência mecânica, ruído e sensibilidade a impacto
SSD	Memória flash e controlador	Baixa latência, silêncio e alta taxa de transferência	Desgaste de escrita e custo por gigabyte maior
Pendrive ou cartão	Memória flash removível	Portabilidade e baixo consumo	Desempenho variável e menor resistência em alguns modelos
Mídia óptica	Leitura por laser	Distribuição e arquivamento em cenários específicos	Baixa capacidade e velocidade em comparação com SSD

Acesso sequencial e acesso aleatório

Acesso sequencial percorre dados em ordem; acesso aleatório alcança posições dispersas. HDDs sofrem mais com acessos aleatórios porque a cabeça precisa se reposicionar. SSDs não possuem partes móveis e apresentam vantagem importante nesse cenário.

Controlador de SSD

O controlador de um SSD distribui gravações entre células, corrige erros e remapeia blocos. Técnicas como wear leveling procuram equilibrar o desgaste. O sistema operacional enxerga um dispositivo de blocos; os detalhes físicos são abstraídos pelo controlador.

Desempenho não depende apenas da interface

Uma interface rápida define um limite de comunicação, mas o dispositivo precisa conseguir produzir ou consumir dados nessa taxa. Filas de solicitações, tamanho dos blocos, padrão de acesso, cache do dispositivo e carga do sistema também influenciam o resultado.

Para Desenvolvimento de Sistemas, a escolha do padrão de acesso é decisiva: muitas gravações pequenas e síncronas podem ser mais caras que operações agrupadas; consultas sem índice podem movimentar muito mais dados do que o necessário.

6. Da unidade de armazenamento até a CPU



Figura 3 — Caminho conceitual percorrido por instruções e dados.

Ao iniciar um programa, o sistema operacional localiza o arquivo executável, reserva espaço de memória e carrega partes necessárias. A CPU busca instruções em endereços de memória; os níveis de cache tentam atendê-las antes de acessar a RAM.

Buffers

Um buffer é uma área temporária usada para acomodar diferenças de velocidade ou de tamanho entre produtores e consumidores de dados. Ao ler um arquivo, o sistema pode transferir um bloco para um buffer na RAM, mesmo que a aplicação solicite poucos bytes por vez.

Memória virtual em uma primeira visão

Programas trabalham com endereços virtuais. O sistema operacional e o hardware traduzem esses endereços para posições físicas. Quando uma parte necessária não está na RAM, o sistema pode buscá-la no armazenamento. Esse mecanismo aumenta a flexibilidade e o isolamento, mas uma quantidade excessiva de faltas de página reduz fortemente o desempenho.

A memória virtual será tratada aqui apenas como ligação entre RAM, armazenamento e execução. O detalhamento de paginação pertence ao estudo de sistemas operacionais.

Exemplo conceitual

1. A aplicação solicita a leitura de um arquivo.
2. O sistema operacional verifica se os dados já estão em memória.
3. Se necessário, o controlador lê blocos do SSD ou HDD.
4. Os dados chegam a um buffer na RAM.
5. A aplicação recebe os bytes solicitados.
6. A CPU processa os dados usando cache e registradores.

Esse fluxo mostra por que mensagens como aguardar entrada e saída não significam que a CPU esteja executando cálculos continuamente. Muitas vezes, a thread fica bloqueada até o dispositivo ou o sistema informar que os dados estão prontos.

7. Dispositivos de entrada e saída

Entrada e saída, ou E/S, reúne os mecanismos que permitem ao computador receber informações, apresentar resultados, armazenar dados e comunicar-se com outros sistemas. Um mesmo equipamento pode exercer mais de uma função.

Categoria	Função	Exemplos
Entrada	Fornecer dados ao sistema	Teclado, mouse, microfone, câmera e sensores
Saída	Apresentar ou atuar a partir de resultados	Monitor, impressora, alto-falante e atuadores
Entrada e saída	Receber e transmitir ou ler e gravar	Tela sensível ao toque, rede, SSD e USB

Periférico, controlador e interface

O periférico é o equipamento que interage com o ambiente. O controlador contém a lógica que adapta o dispositivo ao sistema. A interface estabelece regras elétricas, físicas e lógicas para a comunicação. Em muitos casos, o programador não acessa diretamente esses detalhes: utiliza serviços do sistema operacional e bibliotecas.



Figura 4 — Camadas usuais entre o programa e o periférico.

Registros de um controlador

Registro	Função
Dados	Receber ou fornecer valores transferidos
Estado	Indicar disponibilidade, erro, conclusão ou ocupação
Controle	Selecionar operação, habilitar interrupção ou iniciar transferência

Esses registros podem aparecer em uma região do espaço de endereços, técnica chamada E/S mapeada em memória. Assim, instruções de leitura e escrita acessam tanto memória quanto registradores de dispositivos, conforme o endereço utilizado.

8. Interfaces e comunicação com periféricos

Interfaces padronizam a comunicação e permitem combinar componentes de fabricantes diferentes. Elas definem conectores ou enlases, sinais, formatos de quadros, velocidades, negociação e tratamento de erros.

Interface	Uso comum	Característica relevante
USB	Teclado, mouse, armazenamento e vários periféricos	Conexão serial, alimentação e enumeração de dispositivos
PCI Express	GPU, rede e SSD interno	Enlace serial de alta velocidade com pistas
SATA	HDD e SSD	Interface para dispositivos de armazenamento
HDMI ou DisplayPort	Áudio e vídeo	Transmissão digital para monitores
Ethernet e Wi-Fi	Comunicação em rede	Troca de quadros entre interfaces de rede

Comunicação serial e paralela

Uma interface paralela transmite vários bits simultaneamente por linhas diferentes. Uma interface serial envia bits em sequência por menos linhas e pode atingir taxas elevadas com sincronização e codificação adequadas. Sistemas atuais usam amplamente enlases seriais de alta velocidade.

Taxa nominal e taxa útil

A taxa anunciada por uma interface não corresponde integralmente aos dados da aplicação. Protocolos adicionam cabeçalhos, códigos de controle, confirmações e correção de erros. A taxa útil também depende do dispositivo, do controlador, do driver e do padrão de acesso.

Síncrono e assíncrono no software

Em uma operação síncrona, a chamada aguarda a conclusão antes de prosseguir. Em uma operação assíncrona, o programa inicia a solicitação e continua outras tarefas, recebendo depois um evento, futuro, promessa ou callback. Essa distinção de software não elimina a necessidade de o hardware completar a transferência.

Para aplicações de rede, interfaces gráficas e servidores, evitar bloqueios desnecessários pode melhorar a capacidade de resposta. Entretanto, concorrência assíncrona exige cuidado com ordem, erros, cancelamento e acesso compartilhado a dados.

9. Entrada e saída programada e consulta periódica

Na E/S programada, a CPU executa instruções para consultar o estado do controlador e transferir dados. Uma forma simples é o polling, ou consulta periódica: o programa verifica repetidamente se o dispositivo está pronto.



Figura 5 — No polling, a verificação se repete até o dispositivo estar pronto.

```
enquanto dispositivo_ocupado:  
    estado = ler_registro_de_estado()  
  
    dado = ler_registro_de_dados()  
    processar(dado)
```

Vantagens

- Implementação conceitualmente simples.
- Comportamento previsível em sistemas pequenos ou de tempo real cuidadosamente projetados.
- Pode ser eficiente quando a espera é muito curta e o evento é frequente.

Limitações

- A CPU pode desperdiçar ciclos consultando um dispositivo lento.
- Consultas muito espaçadas aumentam a demora para perceber um evento.
- Muitos dispositivos tornam o laço de verificação mais complexo.

Polling não é sempre incorreto. A decisão depende da frequência do evento, do custo de consulta, do tempo de resposta exigido e das demais tarefas da CPU. Drivers e sistemas embarcados podem combinar polling e interrupções.

Espera ocupada e bloqueio

Na espera ocupada, a CPU continua executando o laço de consulta. Em uma espera bloqueante gerenciada pelo sistema operacional, a thread deixa de disputar a CPU até que o evento ocorra. A segunda abordagem libera processamento para outras tarefas, embora envolva troca de contexto e gerenciamento do escalonador.

10. Interrupções

Uma interrupção permite que o dispositivo sinalize a necessidade de atenção. A CPU não precisa consultar continuamente o controlador. Quando o evento ocorre, o processador termina um ponto seguro da execução, preserva o contexto necessário e transfere o controle para uma rotina de tratamento.



Figura 6 — Sequência conceitual do atendimento de uma interrupção.

Etapas simplificadas

1. O dispositivo altera seu estado e solicita atendimento.
2. O processador reconhece a interrupção conforme prioridade e habilitação.
3. Parte do contexto do programa é preservada.
4. A rotina de tratamento identifica a causa e realiza o trabalho urgente.
5. O evento é reconhecido no controlador e o contexto é restaurado.
6. O programa interrompido continua a partir do ponto adequado.

Interrupção de hardware e exceção não são equivalentes

Interrupções de hardware são normalmente assíncronas em relação à instrução corrente. Exceções, como divisão por zero ou acesso inválido, surgem durante a execução de uma instrução. Sistemas podem usar mecanismos semelhantes para desviar o controle, mas as causas são diferentes.

Prioridade e mascaramento

Quando vários eventos ocorrem, o sistema precisa decidir a ordem de atendimento. Interrupções podem possuir prioridades e algumas podem ser temporariamente mascaradas. O tratador deve ser breve: trabalhos demorados costumam ser adiados para uma etapa posterior do sistema operacional.

Exemplo: ao pressionar uma tecla, o controlador registra o código e gera uma interrupção. O driver recolhe o dado, reconhece o evento e o disponibiliza ao sistema, que então o entrega à aplicação apropriada.

11. Acesso direto à memória

Em transferências de blocos, mover cada palavra por instruções da CPU seria custoso. O acesso direto à memória, ou DMA, permite que um controlador transfira dados entre o dispositivo e a RAM com participação reduzida do processador.



Figura 7 — A CPU prepara a transferência e é avisada quando o bloco termina.

Configuração típica

- Endereço inicial do buffer na memória.
- Quantidade de bytes ou palavras a transferir.
- Direção da transferência: dispositivo para memória ou memória para dispositivo.
- Comando para iniciar a operação.

Método	Participação da CPU	Situação adequada
Polling	Consulta repetidamente o estado e pode mover cada dado	Eventos simples, muito frequentes ou espera curtíssima
Interrupção	Atende quando o dispositivo sinaliza	Eventos esporádicos e transferências pequenas
DMA	Configura o bloco e recebe aviso de conclusão	Grandes blocos de armazenamento, rede, áudio e vídeo

Coerência e sincronização

Como DMA e CPU podem acessar a memória, o sistema deve garantir que ambos enxerguem dados consistentes. Drivers controlam a ordem das operações, usam barreiras quando necessário e tratam a relação com cache. O programador de aplicações normalmente recebe essa complexidade abstraída pelo sistema operacional.

DMA reduz trabalho repetitivo da CPU, mas não torna a transferência instantânea. O barramento, a memória, o controlador e o dispositivo continuam tendo limites de latência e largura de banda.

12. Dois fluxos completos

Exemplo 1 — Pressionamento de uma tecla

1. O teclado detecta a mudança física e gera um código.
2. O controlador disponibiliza o código e sinaliza uma interrupção.
3. O processador desvia temporariamente para o tratador do dispositivo.
4. O driver lê o registro de dados e reconhece a interrupção.
5. O sistema coloca o evento em uma fila e desperta a aplicação interessada.
6. A aplicação processa o caractere ou comando recebido.

Exemplo 2 — Leitura de um arquivo em SSD

1. A aplicação solicita bytes por uma chamada ao sistema.
2. O sistema verifica cache de arquivos e buffers já existentes.
3. Se os dados não estiverem na RAM, o driver programa o controlador do SSD.
4. O controlador localiza os blocos e usa DMA para transferi-los à memória.
5. Uma interrupção informa a conclusão; o sistema valida o resultado.
6. Os bytes são disponibilizados à aplicação, que volta a executar.

Pergunta	Teclado	Leitura de arquivo
Natureza do evento	Poucos bytes e ocorrência irregular	Blocos de dados solicitados
Mecanismo importante	Interrupção	DMA e interrupção de conclusão
Área temporária	Fila de eventos	Buffer ou cache de arquivos
Visão da aplicação	Evento de entrada	Fluxo de bytes ou arquivo

O que a abstração esconde

A chamada de alto nível pode parecer simples, mas envolve camadas de hardware e software. Essa abstração permite portabilidade: o mesmo programa usa uma API semelhante em máquinas cujos controladores e interfaces físicas são diferentes.

Ao diagnosticar lentidão, é útil identificar a camada que espera: aplicação, sistema de arquivos, driver, controlador, interface, dispositivo ou rede. Medir o tempo em cada etapa evita atribuir todo atraso ao processador.

13. Relação com Desenvolvimento de Sistemas

Conhecer memória e E/S ajuda a projetar programas que usam recursos de modo previsível. A arquitetura não determina sozinha a qualidade de uma aplicação, mas explica custos que aparecem em consumo de RAM, tempo de resposta e vazão.

Situação de software	Relação arquitetural	Boa prática
Processar arquivo grande	Dados chegam em blocos pela hierarquia	Usar streaming e buffers adequados
Criar muitos objetos	Aumenta uso de RAM e pressão sobre cache	Evitar retenção desnecessária e medir alocações
Consultar banco sem índice	Mais blocos são lidos e transferidos	Analisar plano de consulta e índices
Bloquear interface gráfica em E/S	Thread espera dispositivo ou rede	Separar trabalho de E/S e atualizar a interface com segurança
Gravar registro a registro	Muitas operações pequenas pagam custos fixos	Agrupar operações quando a consistência permitir
Carregar tudo em memória	Pode causar paginação e falta de RAM	Processar por partes e impor limites

Checklist para investigar desempenho

- O programa está limitado por CPU, memória, armazenamento ou rede?
- O padrão de acesso é sequencial ou aleatório?
- Há muitas operações pequenas que poderiam ser agrupadas?
- A aplicação mantém dados que não precisa mais?
- Threads estão bloqueadas aguardando E/S?
- Existe medição com dados reais antes de otimizar?

Limites desta semana

A cache foi apresentada como nível intermediário e associada à localidade. Na Semana 6, veremos com mais detalhe acertos e falhas, organização em linhas, mapeamento, políticas de substituição e integração dos componentes.

A meta aqui é reconhecer o caminho completo do dado: ele pode estar persistido, ser trazido para a RAM por E/S, aproximar-se da CPU por cache e chegar aos registradores para participar de uma instrução.

14. Exercícios de consolidação

Parte A — Conceitos

1. Explique por que uma única tecnologia não é usada para todos os níveis de memória.
2. Ordene registradores, cache, RAM e SSD da menor para a maior capacidade típica.
3. Diferencie memória volátil de memória não volátil e dê dois exemplos de cada.
4. Por que a SRAM é adequada à cache e a DRAM à memória principal?
5. Diferencie latência de taxa de transferência.
6. Explique por que um arquivo precisa chegar à RAM antes de ser processado diretamente pela CPU.

Parte B — Capacidade e armazenamento

1. Quantas posições podem ser representadas por 10 bits de endereço?
2. Uma memória endereçável por byte possui 16 bits de endereço. Qual é sua capacidade?
3. Compare HDD e SSD em relação a partes móveis e acesso aleatório.
4. Explique por que a taxa útil de uma interface é menor que a taxa nominal.
5. O que é um buffer e por que ele é usado?

Parte C — Entrada e saída

1. Diferencie periférico, controlador e interface.
2. Quais são as funções dos registros de dados, estado e controle?
3. Compare polling e interrupção quanto ao uso da CPU.
4. Descreva as etapas básicas do atendimento de uma interrupção.
5. Explique por que DMA é adequado a transferências de blocos.
6. No fluxo de leitura de um arquivo, indique onde aparecem armazenamento, RAM, DMA e interrupção.

Desafio aplicado

Uma aplicação lê 500 MB de registros, mantém todos em uma lista, ordena o conjunto e grava um resultado pequeno. Proponha duas alterações para reduzir uso de memória ou tempo de E/S. Justifique cada proposta com conceitos desta apostila.

15. Respostas comentadas

Item	Resposta esperada
A1	Velocidade, capacidade e custo entram em conflito; a hierarquia combina tecnologias.
A2	Registradores, cache, RAM e SSD.
A3	Volátil perde conteúdo sem energia; registradores e RAM. Não volátil preserva; SSD e ROM.
A4	SRAM é mais rápida e cara; DRAM oferece maior densidade e menor custo por bit.
A5	Latência é o tempo para iniciar ou concluir um acesso; taxa é o volume transferido por tempo.
A6	A CPU executa instruções e acessa dados em endereços de memória; o armazenamento fornece blocos à RAM.
B1	$2^{10} = 1\,024$ posições.
B2	2^{16} bytes = 65 536 bytes = 64 KiB.
B3	HDD usa mecanismo móvel e sofre mais em acesso aleatório; SSD usa flash e não tem partes móveis.
B4	Protocolos e controles consomem parte da transmissão; dispositivo e software também impõem limites.
B5	Área temporária que acomoda diferenças de velocidade e organiza transferências em blocos.

Item	Resposta esperada
C1	Periférico interage com o ambiente; controlador adapta o dispositivo; interface define regras de comunicação.
C2	Dados transferem valores; estado informa condição; controle seleciona e inicia operações.
C3	Polling consulta repetidamente; interrupção sinaliza a CPU quando há evento.
C4	Sinalização, preservação de contexto, execução do tratador, reconhecimento e retorno.
C5	O controlador move o bloco diretamente entre dispositivo e RAM, reduzindo trabalho repetitivo da CPU.
C6	O arquivo está no armazenamento; DMA o leva a um buffer na RAM; uma interrupção indica conclusão.
Desafio	Possibilidades: processar por blocos, filtrar cedo, ordenar externamente, usar streaming e agrupar gravações.

16. Resumo

- A hierarquia combina níveis rápidos e pequenos com níveis maiores e mais lentos.
- Registradores, cache e RAM são voláteis; armazenamento secundário preserva dados.
- DRAM é usada principalmente na memória principal; SRAM é comum em cache.
- Endereços identificam posições e n bits permitem representar 2^n combinações.
- HDD, SSD e flash apresentam diferentes custos, latências e características físicas.
- Controladores ligam periféricos ao sistema por interfaces padronizadas.
- Polling consulta o estado; interrupções notificam eventos; DMA transfere blocos com menor participação da CPU.
- Programas eficientes consideram buffers, padrão de acesso, bloqueio e consumo de memória.

Glossário

Termo	Definição
DMA	Transferência direta entre controlador e memória com participação reduzida da CPU.
Driver	Software do sistema operacional que controla um dispositivo.
Firmware	Software armazenado em memória não volátil que controla ou inicializa hardware.
Latência	Tempo entre solicitar uma operação e obter o resultado ou iniciar a transferência.
Polling	Consulta periódica ao estado de um dispositivo.
Taxa de transferência	Quantidade de dados movimentada por unidade de tempo.
Volátil	Memória que perde o conteúdo quando deixa de receber energia.

Referências

PATTERSON, David A.; HENNESSY, John L. Organização e Projeto de Computadores: a interface hardware/software. Rio de Janeiro: Elsevier, 2017.

STALLINGS, William. Arquitetura e Organização de Computadores. São Paulo: Pearson, 2017.

TANENBAUM, Andrew S.; AUSTIN, Todd. Organização Estruturada de Computadores. São Paulo: Pearson, 2013.

WEBER, Raul Fernando. Fundamentos de Arquitetura de Computadores. Porto Alegre: Bookman, 2012.

Material elaborado para a disciplina Arquitetura e Organização de Computadores — IF Sudeste MG, 2026.